Day 1 – Environment & Project Setup

🏷 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
    - Ensure you have Python 3.12+ installed.
    - Open a clean terminal window in an empty project directory.
    - Have a web browser open and ready to navigate to `http://127.0.0.1:8000/`.
2. **Prerequisites Checklist**:
    - Students should have standard terminal environments working on their OS (Mac/Linux or Windows Git Bash/PowerShell).
3. **Required Material**:
    - Whiteboard or slide showing the end-to-end system architecture (User → Orders API → DB/GameKey → Expiration Engine → Celery → Webhook Queue → Publisher).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Install and configure modern Python package management using `uv`.
- Initialize and activate a virtual environment.
- Scaffold a new Django project and custom app.
- Securely configure environment variables using `python-decouple`.
- Initialize a Git repository and prevent secrets leaks using `.gitignore`.
- Run the Django development server and verify the default setup.

⏱ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Interactive overview of the bootcamp outcome (game key expiry & publisher webhooks).
2. Key Concepts & Core Ideas	15 min	Introduction to MTV, API backends, package management (`uv`), and configuration security.
3. Live Coding Walkthrough	45 min	Live setup of python environment, Django, and decoupling configuration.
4. Check for Understanding	10 min	Q&A on virtual environments, secrets management, and DEBUG mode.
5. Class Wrap-up & Checkpoint	5 min	Verification of students' local environments.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Open a diagram showing the complete workflow. Explain the business problem: digital game key marketplaces need to deliver keys to users, track their expirations, and automatically notify publishers when a key expires so they can deactivate it on their servers.
- **Big Picture**: Show the finished flow: User buys key → key expires → Celery fires webhook → publisher receives it. Explain to students: *Make them care about the outcome before writing a single line.*

2. Key Concepts & Core Ideas (15 min)

Introduce the following definitions to the students:

- **What is Django?** MTV (Model-Template-View) framework. Mention that we'll use it purely as a REST API backend with no HTML templates.
- **What is DRF?** Django REST Framework. A toolkit built on top of Django that gives us serializers, viewsets, and routers for building RESTful APIs.
- **Virtual environments**: Why isolation matters (preventing dependency conflicts and ensuring reproducibility across developers' machines).
- **DEBUG = True vs False**: Django displays highly detailed error pages containing traceback information. Explain that this is crucial for development but a critical security leak in production.
- **Why python-decouple?** Reading from `os.environ.get()` is functional but `decouple` simplifies casting types (e.g. converting a string `"True"` to boolean `True`) and handling defaults and local `.env` files in a clean API.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Tooling Installation & Scaffolding

- **Explain**: Contrast standard `pip + venv` with `uv`. Why are we using `uv`? Speed, lockfiles, and reproducibility. Run the installation and scaffolding commands while students follow along.
- **Code**:

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
uv venv
source .venv/bin/activate

# Add dependencies
uv add django djangorestframework python-decouple celery redis pyte

# Scaffold project and app
django-admin startproject gamekey_platform .
python manage.py startapp games
```

- **Verify**: Confirm that every student sees `(.venv)` in their command line prompt. Pause here to ensure everyone is inside the active virtual environment before moving on.

- ⚠ **Common Pitfalls**:
  - **Forgetting to activate the venv**: If students skip `source .venv/bin/activate`, they will install dependencies globally or get "django not found". Teach them to run `which python` to verify the path points to `.venv`.
  - **Windows path differences**: Flag that on Windows, the activation command is `.venv\Scripts\activate` rather than `.venv/bin/activate`.

Step 3.2: Environment Variable Configuration

- **Explain**: Explain why secrets (such as the Django `SECRET_KEY`) should never go into version control (source code). Show what happens if you push a secret key to a public GitHub repository. Create the local environment file.
- **Code**: Create `.env` in the project root directory:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

- **Verify**: Check that the `.env` file is created and contains the correct key-value pairs.

Step 3.3: Django settings.py Decoupling

- **Explain**: Now we need to modify Django's settings to read our newly created `.env` file rather than having hardcoded variables.
- **Code**: Update `gamekey_platform/settings.py` to import and use `python-decouple`:

```
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
```

- **Verify**: Run the development server to verify the configuration is loaded correctly:

```
# Verify setup
python manage.py runserver
```

  Navigate to `http://localhost:8000/` and verify the Django welcome page loads.
- ⚠ **Common Pitfalls**:
  - **`SECRET_KEY` still hardcoded**: Students might add `python-decouple` but forget to update the actual variables in `settings.py`. Test this by removing the key from `.env` and seeing if Django throws an error on startup.

Step 3.4: Git Initialization

- **Explain**: Set up version control. Introduce the `.gitignore` file and emphasize that the `.env` file must be ignored immediately to prevent leaking the secret keys.
- **Code**:

```
git init
```

  Create `.gitignore` and include `.env` (along with standard Python/Django patterns: `__pycache__`, `.venv`, `*.pyc`, etc.):

```
.env
.venv/
*.pyc
__pycache__/
db.sqlite3
```

- **Verify**: Run `git status` and confirm that `.env` is NOT listed under untracked files.
- ⚠ **Common Pitfalls**:
  - **Committing `.env` to Git**: Show the students what `git status` looks like *before* and *after* editing `.gitignore` to show how Git tracks files.

---

4. Check for Understanding (10 min)

Question: "Why can't we just hard-code `SECRET_KEY = 'abc123'` in settings.py?"

Answer: Hard-coding `SECRET_KEY` in source code poses a severe security risk. If the repository is pushed to a public platform (like GitHub), anyone can see the key. Attackers can use it to forge signed cookies, session data, or password reset tokens. Storing it in an environment variable (`.env` file) keeps it private and allows different values for development and production environments.

Question: "What would happen if two projects on the same machine installed different versions of `django` without virtual environments?"

Answer: Installing different versions globally would cause a conflict because Python can only resolve one version of a package in its global site-packages directory. Installing a different version for the second project would overwrite the version needed by the first project, breaking it. Virtual environments isolate dependencies per project, allowing each to run its required version independently.

Question: "What does `DEBUG=True` change about how Django behaves?"

Answer: When `DEBUG=True`, Django displays detailed error traceback pages to the client (useful for debugging but a huge security leak in production because it exposes database queries, settings, and path variables). It also runs the built-in development server's auto-reloader and serves static/media files automatically. In production (`DEBUG=False`), it returns a generic 500 error page, requires `ALLOWED_HOSTS` to be set, and disables automated static file serving.

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - Student shows the Django welcome rocket page running locally at `http://127.0.0.1:8000/`.
  - The `.env` file exists and contains `SECRET_KEY` and `DEBUG`.

- ○ Running `git status` shows `.env` is untracked (not staged) and excluded.
- ○ Running `git log --oneline` shows at least one commit.